

EcoTwin Technical Implementation Specification (TIS)

Implementation Blueprint

June 12, 2026

Contents

1 Purpose	2
2 Repository Structure	2
3 Frontend Structure	3
4 Backend Structure	3
5 ML Structure	4
6 Data Pipeline Structure	4
7 Coding Standards	5
7.1 Python	5
8 Frontend Standards	5
9 Region Generation Algorithm	6
9.1 Objective	6
9.2 Input	6
9.3 Output	6
9.4 Algorithm	6
10 Earth Engine Pipeline	6
10.1 Datasets	6
10.2 Cloud Masking	7
10.3 Composite Generation	7
11 Feature Extraction Workflow	7
12 Prithvi Workflow	8
12.1 Purpose	8
12.2 Input	8
12.3 Preprocessing	8
12.4 Inference	8
13 Embedding Generation Job	9
14 pgvector Schema	9
15 Similarity Search Service	10

16 Forecasting Engine	10
16.1 Approach	10
16.2 Workflow	10
16.3 Forecast Metrics	11
17 Background Jobs	11
18 API Versioning	12
19 Environment Variables	12
20 Docker Architecture	12
21 Docker Compose Layout	13
22 Nginx Configuration	13
23 CI/CD Pipeline	13
24 Testing Strategy	14
24.1 Backend	14
24.2 Frontend	14
25 Logging Strategy	15
26 Monitoring	15
27 Production Infrastructure	15
28 Release Roadmap	16
29 Definition of Done	16
30 Conclusion	16

1 Purpose

This document defines the detailed implementation specifications required to build EcoTwin.
It serves as the engineering execution guide for:

- Backend Developers
- Frontend Developers
- GIS Engineers
- ML Engineers
- DevOps Engineers

The document includes:

- Folder Structure
- Coding Standards
- Earth Engine Workflow
- Region Generation Logic
- Prithvi Integration
- Vector Search Design
- Forecasting Workflow
- Docker Deployment
- CI/CD Pipeline

2 Repository Structure

ecotwin/

frontend/

backend/

ml/

data_pipeline/

infrastructure/

docs/

scripts/

tests/

docker/

3 Frontend Structure

frontend/

src/

app/

pages/

HomePage

RegionPage

ForecastPage

ReportPage

components/

Map

RegionCard

SimilarityPanel

ForecastPanel

Charts

ReportViewer

hooks/

services/

types/

utils/

assets/

4 Backend Structure

backend/

app/

api/

core/

models/

schemas/

services/

- repositories/
- database/
- middleware/
- jobs/
- workers/
- main.py

5 ML Structure

- ml/
 - embeddings/
 - forecasting/
 - feature_extraction/
 - similarity/
 - training/

6 Data Pipeline Structure

- data_pipeline/
 - region_generation/
 - sentinel_ingestion/
 - cloud_masking/
 - composites/
 - exports/
 - batch_jobs/

7 Coding Standards

7.1 Python

Rules:

- Python 3.11+
- PEP8 Compliance
- Type Hints Mandatory
- Docstrings Mandatory
- Black Formatter
- Ruff Linter

Example:

```
def get_region(
    region_id: str
) -> Region:
    """
    Retrieve region metadata.
    """
    pass
```

8 Frontend Standards

- TypeScript Strict Mode
- Functional Components
- React Hooks
- ESLint
- Prettier

Component Naming:

RegionCard.tsx

ForecastPanel.tsx

SimilarityPanel.tsx

9 Region Generation Algorithm

9.1 Objective

Create fixed-size ecosystem regions.

Region Size:

[5km $\times$ 5km]

9.2 Input

Project Boundary Polygon

9.3 Output

Region Grid

9.4 Algorithm

1. Load boundary polygon.
2. Convert to projected CRS.
3. Generate square grid.
4. Clip to boundary.
5. Store centroid.
6. Assign region ID.

Output:

AP_000001

AP_000002

AP_000003

10 Earth Engine Pipeline

10.1 Datasets

Primary:

COPERNICUS/S2_SR_HARMONIZED

Secondary:

COPERNICUS/S2_CLOUD_PROBABILITY

10.2 Cloud Masking

Apply:

- Cloud Probability
- SCL Filtering

Remove:

- Clouds
- Cirrus
- Cloud Shadows
- Snow

10.3 Composite Generation

Method:

Median Composite

Period:

Yearly

2018-2025

Output:

- RGB Composite
- Feature Composite

11 Feature Extraction Workflow

For each region:

Compute:

- NDVI
- NDWI
- NBR

Store:

- Mean
- Std
- Min
- Max

12 Prithvi Workflow

12.1 Purpose

Generate ecosystem embeddings.

12.2 Input

Region Composite

Bands:

B2
B3
B4
B8
B11
B12

12.3 Preprocessing

1. Normalize bands.
2. Resize to model input size.
3. Convert to tensor.

12.4 Inference

Composite

|
V

Prithvi Encoder

|
V

Embedding

Output:

768 Dimensions

Store:

region_embeddings

table.

13 Embedding Generation Job

Batch Process:

For Each Region

For Each Year

Load Composite

Generate Embedding

Save Vector

Schedule:

Weekly

14 pgvector Schema

Enable:

```
CREATE EXTENSION vector;
```

Table:

```
CREATE TABLE region_embeddings(  
  region_id UUID,  
  year INTEGER,  
  embedding VECTOR(768)  
);
```

Index:

```
CREATE INDEX idx_region_embedding  
ON region_embeddings  
USING ivfflat  
(embedding vector_cosine_ops);
```

15 Similarity Search Service

Workflow:

Selected Region

|
V

Embedding

|
V

pgvector Search

|
V

Top-K Analogs

Query:

```
SELECT *
```

```
FROM region_embeddings
```

```
ORDER BY embedding  
<=> query_embedding
```

```
LIMIT 10;
```

16 Forecasting Engine

16.1 Approach

Analog Forecasting

16.2 Workflow

Current Region

|
V

Find Historical Analogs

```
...  
  |  
  V  
...
```

Retrieve Future Trajectories

```

  ...
  |
  V
  ...

```

Average Changes

```

  ...
  |
  V
  ...

```

Forecast

16.3 Forecast Metrics

- NDVI
- NDWI
- NBR

Forecast Horizon:

- 1 Year
- 3 Years
- 5 Years

17 Background Jobs

Use:

Celery

Redis

Jobs:

- Composite Generation
- Embedding Generation
- Forecast Cache Refresh
- Report Generation

18 API Versioning

Pattern:

`/api/v1/`

Future:

`/api/v2/`

19 Environment Variables

Backend:

`DATABASE_URL=`

`REDIS_URL=`

`JWT_SECRET=`

`GEE_SERVICE_ACCOUNT=`

`AWS_BUCKET=`

Frontend:

`VITE_API_URL=`

20 Docker Architecture

Containers:

`frontend`

`backend`

`postgres`

`redis`

`nginx`

`worker`

21 Docker Compose Layout

services:

frontend

backend

postgres

redis

worker

nginx

22 Nginx Configuration

Responsibilities:

- Reverse Proxy
- SSL Termination
- Compression
- Static Asset Serving

Routes:

/api/*

backend

/

frontend

23 CI/CD Pipeline

Tool:

GitHub Actions

Pipeline:

Push

|

Lint

|

Unit Tests

|

Build

|

Docker Build

|

Deploy

24 Testing Strategy

24.1 Backend

Framework:

pytest

Coverage Target:
[80]

24.2 Frontend

Framework:

Vitest

React Testing Library

Coverage:
[80]

25 Logging Strategy

Use:

Structlog

Levels:

- INFO
- WARNING
- ERROR
- CRITICAL

Store:

logs/

directory.

26 Monitoring

Metrics:

- API Latency
- Search Latency
- Forecast Latency
- CPU
- Memory

Stack:

Prometheus

Grafana

Loki

27 Production Infrastructure

Minimum Production Server:

CPU	8 Core	heightRAM
16 GB	heightStorage	200 GB SSD
heightOS	heightOS	
Ubuntu 24.04	height	

28 Release Roadmap

Phase 1

- Grid Generation
- Sentinel Pipeline
- Database Setup

Phase 2

- Prithvi Integration
- Similarity Search
- Forecast Engine

Phase 3

- Frontend
- Reporting
- Deployment

29 Definition of Done

A feature is complete when:

- Code Reviewed
- Unit Tested
- Integrated
- Documented
- Deployed to Staging
- Approved

30 Conclusion

This specification provides the detailed implementation blueprint for EcoTwin.

It defines the complete technical structure required to implement ecosystem discovery, similarity search, temporal analysis, analog forecasting, and production deployment.